



AI 스타트업 투자 평가 RAG 설계서

문서 정보

항목	내용
캠퍼스 / 반	울산캠퍼스 / 4반
팀원	박주윤, 배영환, 배은빈, 윤도균, 이경민
작성일	2026-09-29 (개정 2026-09-30, 코드 반영)
프로젝트 주소	https://github.com/songjung-good/ai_analysis.git
기준 코드	master 브랜치 (2026-09-30)

개정 요약

2026-09-30 master 코드와 대조한 결과, 임베딩 검증은 market_docs에 반영됐지만 State에 추가한 보완 재실행 키(`revision_request` , `revision_count`)와 `selection_conditions` 는 코드에 없다. 이번 개정은 코드를 기준으로 설계서를 고치고, 코드에 없는 설계는 후속 과제로 옮겼다.

항목	이전 설계서	현재 코드	이번 개정
보완 재실행 (<code>revision_request</code> , <code>revision_count</code>)	State 키 + 조건 분기 ①	없음	State-D-3에서 제외, 후속 과제로 이동
<code>selection_conditions</code>	별도 State 키	없음 (탐색 조건은 <code>criteria</code> 로 전달)	<code>criteria</code> 에 통합
<code>decision</code> 값	추천 / 조건부 검토 / 보류	<code>invest</code> / <code>conditional</code> / <code>hold</code>	영문 값으로 수정
조건부 검토 처리	보고서 생성	남은 후보가 있으면 다음 후보 평가	코드 기준으로 수정
근거 부족 항목	1점 처리 후 필수 보류	점수 <code>null</code> + 필수 보류 (6개 항목 모두)	코드 기준으로 수정
평가 횟수 기준	<code>len(evaluations)</code>	<code>current_idx + 1 < max_iterations</code>	코드 기준으로 수정
후보 없음	정의 없음	탐색 → 보고서 생성 분기	D-3에 추가
시장 Retriever	BM25 + Dense MMR	BM25 + Dense similarity	변경 사유와 함께 수정
임베딩 검증 (B-3)	방법만 기술	market_docs 30문항 평가 완료	결과 추가, tech_docs는 미실시로 표기
<code>references</code> 형식	출처 ID, 제목, URL/페이지, 기업명, 사용 에이전트	<code>Evidence</code> (source_id, title, url, page, excerpt, published_at)	코드 기준으로 수정
<code>report</code>	최종 투자 보고서(str)	생성된 PDF의 절대 경로	코드 기준으로 수정
기술·제품 검증	Crag 개요만 기술	구현 완료 (기업 근거 폴백 포함)	상세 섹션(F) 추가

변경한 곳은 각 섹션 제목에 (개정)으로 표시했다.

A. Domain 선정

선정 도메인

- 도메인: Physical AI·Robotics
- 선정 이유:
 - 시장 또는 사회적 필요: 고령화와 기피 현상으로 제조업과 물류 현장의 인력이 부족하고, 위험·반복 작업의 자동화 수요가 커지면서 로봇 도입 논의가 진행 중이다.

- **AI 적용 가능성:** 생성형 AI와 로봇 파운데이션 모델의 발전으로 AI 산업이 Physical AI로 확장되고 있다. 기존에 자동화가 어려웠던 비정형 작업까지 대상이 넓어지면서 기술 차별성을 가진 스타트업이 다수 등장했다.
- **투자 관점의 매력:** 제조·물류를 포함해 산업 전반으로 확장될 수 있는 큰 시장이며, 스타트업이 활발히 등장해 후보군을 구성하기 좋다. 하드웨어 양산, 원가, 안전 인증, 유지보수 같은 도메인 고유 리스크가 뚜렷해 일반 소프트웨어 스타트업과 다른 판단 기준을 설계할 의미가 있다.

대상 스타트업 조건

- 비상장 기업
- 투자 단계: Seed ~ Series C
- M&A 등 Exit이 완료되지 않은 기업
- 최근 2년 내 투자나 활동 기록이 있을 것
- 공개 자료로 평가 가능할 것

B. 설계

B-1. 에이전트 정의 (개정)

입력·출력은 각 Agent가 실제로 읽고 쓰는 State 키 기준으로 고쳤다. 쓰기 권한은 `contracts.py`의 `guard_node`가 실행 중 검사한다.

에이전트	역할	입력	출력 (State 키)	사용 도구	RAG	구현 파일
1) 스타트업 탐색	선정 조건에 맞는 후보 기업과 기본 정보 수집	<code>domain</code> , <code>criteria</code> (지역·허용 투자 단계·후보 수 상한)	<code>candidates</code> , <code>current_idx</code> , <code>selected_startup</code> , <code>references</code>	<code>web_search</code> (Tavily)	X	<code>agents/discovery.py</code>
2) 분야·고객 분류	세부 분야와 실제 비용을 지불하는 고객 정의	<code>selected_startup</code>	<code>profile</code> (<code>subdomain</code> , <code>paying_customer</code> , <code>customer_problem</code>), <code>references</code>	Tool 없음, LLM 구조화 응답	X	<code>agents/profile.py</code>
3) 기술·제품 검증	AI의 인식·판단·행동 기능과 기술 차별성 분석	<code>selected_startup</code> , <code>profile</code> , <code>tech_docs</code>	<code>technical_analysis</code> , <code>references</code>	<code>search_tech_docs</code> , <code>web_search</code> (fallback), LLM 관련성 평가	O	<code>agents/technical.py</code>
4) 현장 도입·사업성	시연·시험 운영·유료 운영을 구분하고 고객 도입 효과 분석	<code>selected_startup</code> , <code>profile</code>	<code>business_analysis</code> , <code>references</code>	<code>web_search</code>	X	<code>agents/business*.py</code>
5) 시장성·경쟁	세부 분야 수요와, 같은 고객 문제를 푸는 경쟁사 비교	<code>selected_startup.name</code> , <code>profile.subdomain</code> (필수), <code>paying_customer</code> , <code>customer_problem</code>	<code>market_analysis</code> , <code>references</code>	<code>search_market_docs</code> , <code>web_search</code> (fallback), 공용 CRAG	O	<code>agents/market.py</code> , <code>crag.py</code>
6) 투자 판단	평가표 기준으로 6개 항목 채점, 필수 보류 확인	세 분석 결과, <code>profile</code> , <code>references</code>	<code>scores</code> , <code>investment_score</code> , <code>decision</code> , <code>decision_reason</code> , <code>evaluations</code>	<code>calculate_score</code> (Python)	X	<code>agents/decision.py</code> , <code>scoring.py</code>
7) 보고서 생성	분석 결과를 5페이지 PDF 투자 보고서로 작성	전체 State	<code>report</code> (PDF 절대 경로)	ReportLab, PyMuPDF (페이지 검증)	X	<code>agents/report.py</code>

- Tool 권한은 `tools/registry.py`의 `tools_for(agent)`가 Agent별로 필요한 것만 준다.
- 모든 LLM은 ChatOpenAI이며 모델명은 `OPENAI_MODEL` 환경변수로 주입한다.

B-2. RAG 적용 대상 (개정)

기술 쪽은 실제 적재한 문서·페이지를, 시장 쪽은 검색 평가 후 바뀐 Retriever를 반영했다.

항목	3) 기술·제품 검증	5) 시장성·경쟁
적용 에이전트	기술·제품 검증 에이전트	시장성·경쟁 에이전트
적용 목적	최신 정보 반영: VLA, 로봇 파운데이션 모델 등 최신 기술 개념은 LLM 사전 지식이 부정확하거나 오래됐을 수 있음. 환각 방지: 논문·기술 보고서에 근거해 기업 기술의 수준과 한계를 검증	정확한 정보 반영: 시장 규모, 성장률, 투자 동향 같은 수치는 출처가 명확해야 함. 공신력 있는 산업 리포트로 시장 기회와 경쟁 구도 판단
데이터 출처	SPRi 「피지컬 AI의 현황과 시사점」 (IS-202) PDF 8-15, 37-38 쪽 (주요 기술, 유형, 기술적 한계·비용) LingBot-VLA (arXiv 2601.18692) 본문 1-9쪽 τ0-VLA (arXiv 2608.16885) 본문 1-11쪽	스타트업얼라이언스 「2026 피지컬 AI 스타트업맵」, SPRi 「피지컬 AI의 현황과 시사점」, IFR World Robotics 2025 발표자료
문서 분량	30쪽, 청크 157개 (기준 70쪽 이내)	약 130쪽 이내
원문 관리	<code>data/tech_docs/</code> + <code>manifest.json</code> (제목·URL·발행일·사용 페이지)	<code>data/market/sources.json</code> + 수작업 전사본 <code>data/market/transcripts/</code> (텍스트 추출이 깨진 페이지 보완)
Document Loader	PyMuPDFLoader	PyMuPDFLoader + 전사본 병합
Text Splitter	RecursiveCharacterTextSplitter	RecursiveCharacterTextSplitter, 섹션 제목(## ~ ####)에서 먼저 분할
Chunk 설정	<code>chunk_size=800, chunk_overlap=100</code>	<code>chunk_size=1000, chunk_overlap=150</code>
Vector Store	Chroma (collection: <code>tech_docs</code>)	Chroma (collection: <code>market_docs</code>)
Retriever	MMR	EnsembleRetriever (BM25 + Dense similarity) (개정)
검색 설정	<code>top_k=5, fetch_k=20, lambda_mult=0.5</code>	<code>top_k=5, weights=[0.4, 0.6]</code> (BM25 : Dense)
적재 명령	<code>python scripts/ingest_tech_docs.py</code>	<code>python -m ai_investment.ingest market</code>

- **시장 Retriever 변경:** Dense MMR은 정답 청크 대신 같은 페이지의 다른 청크를 올려 Recall@5가 0.633에 그쳤다. similarity로 바꾸자 0.833, 섹션 경계 분할까지 적용해 0.867이 됐다 (`docs/MARKET_RETRIEVAL_EVAL.md`).
- 기술 문서에서 References·Appendix는 검색 잡음을 줄이려고 제외했다. SPRi는 본문 쪽번호보다 PDF 쪽번호가 5 크다.
- 두 VLA 논문은 arXiv 프리프린트이므로 보고서에 인용할 때 프리프린트임을 표기한다.

B-3. Embedding 모델 선정 (개정: 검증 결과 추가)

`nlpai-lab/KURE-v1` 을 유지한다. `market_docs` 30문항 검증에서 한국어 Recall@5가 0.900으로 공동 1위였고, 뒤집을 근거가 없었다.

후보 비교

후보 모델	한국어 성능 (Recall@1 평균)	임베딩 차원	최대 입력	속도	메모리·비용	라이선스	비고
nlpai-lab/KURE-v1	0.5264	1024	8192 토큰	보통 (bge-m3와 동일 구조)	약 568M 파라미터, 로컬 실행 무료	MIT	고려대 NLP&AI 연구실, bge-m3를 한국어로 추가 학습
BAAI/bge-m3	0.5178	1024	8192 토큰	보통	약 568M 파라미터, 로컬 실행 무료	MIT	100개 이상 언어, dense·sparse·multi-vector 동시 출력
intfloat/multilingual-e5-large	0.5005	1024	512 토큰	보통	약 560M 파라미터, 로컬 실행 무료	MIT	<code>query:/passage:</code> 접두어 필수

최종 선정

- **선정 모델:** `nlpai-lab/KURE-v1` (`tools/rag.py` 의 `KUREEmbeddings` , 정규화 임베딩)
- **선정 이유:**
 - **한국어 지원:** bge-m3를 약 200만 개의 한국어 질문·문서 쌍으로 추가 학습한 모델로, 한국어 보고서와 영어 논문을 번역 없이 함께 처리한다.
 - **실행 환경:** 1024차원, 최대 8192 토큰이라 800~1000자 청크를 잘림 없이 처리한다. `multilingual-e5-large`는 512 토큰을 넘으면 잘린다.
 - **비용:** MIT 라이선스로 로컬에서 무료 실행한다.

검증 방법 (개정: 실제 수행 방식)

- **평가 세트**: 200자 이상 청크에서 한국어 20개, 영어 10개를 무작위 추출(seed 42)하고, LLM이 청크마다 50자 이내 한국어 질문을 생성했다 (data/market/eval/questions.json).
- **정답 판정**: 청크 ID 대신 정답 청크 문장의 절반 이상을 담은 검색 결과를 정답으로 본다(content 지표). 청크 ID는 분할 설정을 바꾸면 다른 본문을 가리키기 때문이다.
- **지표와 그룹**: Recall@5, MRR@5를 ① 한국어→한국어(ko), ② 한국어→영어(en), ③ 전체(all)로 나눠 측정했다.
- **정성 평가**: 도메인 질문 5개의 정답 페이지를 PDF에서 확인해 두고, 운영 검색 상위 5개 안의 순위를 코드로 채점한다.
- **실행**: `python -m ai_investment.retrieval_eval score market --models kure bge-m3 e5`

검증 결과 (market_docs, dense 검색 기준)

모델	ko R@5	en R@5	all R@5	all MRR@5
KURE-v1	0.900	0.800	0.867	0.676
bge-m3	0.850	0.800	0.833	0.649
multilingual-e5-large	0.900	0.400	0.733	0.537
KURE-v1 hybrid (운영 설정)	0.900	0.800	0.867	0.671

- **판정**: 판정 기준인 ko R@5에서 KURE-v1과 e5가 0.900으로 같고, 분할 방식을 바꾸면 bge-m3가 앞서기도 했다(20문항 중 1문항 차이). 교체할 근거가 없어 KURE-v1을 유지한다. e5는 영어 문서에서 0.400으로 크게 떨어진다.
- **정성 평가**: 도메인 질문 5개 모두 상위 5위 안에서 정답 페이지를 찾았다(1위 3개, 2위 1개, 3위 1개).
- **한계**: 30문항이라 0.05 차이는 순위를 확정하지 못한다. BM25는 한국어 질문→영어 문서에서 R@5 0.200으로 거의 작동하지 않는다.
- tech_docs 검증은 아직 수행하지 않았다 (후속 과제).

C. 투자 판단 기준

평가표

가중치 합계는 100%이며, 코드(scoring.py 의 WEIGHTS)의 항목 키를 함께 적었다.

예) 카본식스

평가 항목	코드 키	가중치	평가 기준	점수 범위	평가 점수	근거
창업자·팀	team	30%	전문성, 실행력, 커뮤니케이션	1~5	5	문태연 대표는 2019년 코그넥스에 2300억 원에 인수된 수아랩의 초기 멤버이자 사업총괄 부대표 출신 (유니콘팩토리). 시드 투자사가 글로벌 엑싯 경험을 보유한 창업팀으로 평가 (바이라인네트워크, 2025.05). CEO·CTO·CHO·CPO 경영진 체계 (헤럴드경제, 2026.07)
시장성	market	25%	시장 규모, 성장 가능성, 수요	1~5	4	로봇 시장은 2030년 1853억 달러, 연평균 약 20.3% 성장 전망 (딜사이트, 모더 인텔리전스 인용). 제조 현장 자동화 수요 확인
제품·기술력	technology	15%	독창성, 구현 가능성	1~5	4	2025년 9월 출시한 '시그마킴'은 AI SW와 HW를 결합한 솔루션으로, 작업자가 센서 글러브를 끼고 한 번 시연하면 AI가 학습해 로봇이 자율 수행 (머니투데이, 2026.07)
경쟁 우위	competition	10%	진입장벽, 특허, 차별성	1~5	3	현장 작업 데이터가 AI 모델을 개선하는 선순환 구조 (헤럴드경제, 2026.07). 특허 현황 미확인
실적	traction	10%	매출, 계약, 사용자 수	1~5	4	리드 투자사가 실제 제조 현장에 제품을 배치하고 매출로 연결한 기업이라고 평가 (유니콘팩토리, 2026.07). 매출 규모 비공개
투자 조건	investment_terms	10%	기업가치, 자본율, 투자 단계	1~5	3	2024년 7월 설립, 약 600억 원 시리즈A (DSC·LB인베스트먼트 공동 리드). 누적 투자금 약 684억 원 (머니투데이). 기업가치 비공개
합계		100%			4.10	투자 추천 (invest)

채점 방식 (개정)

- LLM이 항목마다 1~5 정수 점수, 근거 문장(확인된 사실 → 기준 연결 → 점수 선택 이유 → 한계), source_ids 를 반환한다. 숫자 점수에는 출처가 반드시 있어야 한다.

- 판단에 필요한 정보가 없으면 점수를 강제로 매기지 않고 `null` 로 남긴다. 자료 부재를 1점이나 중립점으로 대체하지 않는다.
- 가중합과 최종 판단은 LLM이 아니라 Python(`calculate_score`)이 계산한다.
- 업계 일반 논문의 성능을 기업 성능으로 쓰지 않는다. 기술 Agent의 `benchmark_context` 가 이 구분을 제공한다.

최종 판단 규칙 (개정)

조건	decision	표기
가중 점수 4.0 이상, 필수 보류 조건 없음	invest	투자 추천
가중 점수 3.0 이상 4.0 미만, 필수 보류 조건 없음	conditional	조건부 검토
가중 점수 3.0 미만, 또는 필수 보류 조건 해당	hold	보류

필수 보류 조건 (점수와 무관하게 `hold`):

- **정보 부족**: 6개 항목 중 하나라도 `null` 이거나, LLM이 `critical_information_gaps` 를 기록한 경우. 항목이 비면 가중 점수도 계산하지 않는다(`None`).
- **분석 누락**: 기술·사업성·시장 분석 중 하나라도 비어 있거나, 분석에 연결된 출처가 전혀 없는 경우 (LLM 호출 없이 보류).
- **중대한 법률·안전 리스크**: `blocking_risks` 에 `legal` 또는 `safety` 범주가 출처와 함께 기록된 경우. 인증 정보 미공개만으로 위법으로 단정하지 않는다.
- **대상 조건 위반**(상장, Exit 완료, 상장 예비심사 청구)은 투자 판단이 아니라 스타트업 탐색 단계에서 후보를 제외하는 방식으로 처리한다.

D. 그래프 설계

D-1. State 설계 (개정)

`state.py` 의 `GraphState` 기준 17개 키다. 이전 설계서의 `selection_conditions`, `revision_request`, `revision_count` 는 코드에 없어 제외했다.

State 키	타입	생성·수정 Agent	Reducer	설명
<code>domain</code>	<code>str</code>	초기 입력	없음	선정 도메인 (Physical AI/Robotics)
<code>criteria</code>	<code>dict</code>	초기 입력	없음	탐색 조건 (지역, 허용 투자 단계, 후보 수 상한). 평가 가중치는 <code>scoring.WEIGHTS</code> 에 고정
<code>candidates</code>	<code>list[Startup]</code>	스타트업 탐색	없음 (덮어 쓰기)	후보 기업 목록
<code>current_idx</code>	<code>int</code>	스타트업 탐색, 다음 후보 선택	없음	현재 평가 중인 후보 인덱스
<code>max_iterations</code>	<code>int</code>	초기 입력	없음	최대 평가 기업 수 (1 이상). <code>current_idx + 1 < max_iterations</code> 일 때만 다음 후보로 이동
<code>selected_startup</code>	<code>Startup</code>	스타트업 탐색, 다음 후보 선택	없음	평가 대상 기업 (<code>name</code> , <code>product</code> , <code>funding_stage</code> , <code>team</code>)
<code>profile</code>	<code>dict</code>	분야·고객 분류	없음	<code>subdomain</code> , <code>paying_customer</code> , <code>customer_problem</code> . 다음 후보에서 다시 덮어씀
<code>technical_analysis</code>	<code>dict</code>	기술·제품 검증	없음	<code>summary</code> , <code>core_technologies</code> , <code>differentiation</code> , <code>performance_evidence</code> , <code>benchmark_context</code> , <code>limitations</code> , <code>evidence_level</code> , <code>missing_items</code> , <code>startup_name</code> , <code>retrieval</code>
<code>business_analysis</code>	<code>dict</code>	현장 도입·사업성	없음	<code>commercialization_stage</code> , <code>stage_reason</code> , <code>customer_cases</code> , <code>deployment_costs</code> , <code>deployment_effects</code> , <code>regulatory_risks</code> , <code>evidence_assessment</code> , <code>information_gaps</code> , <code>summary</code>
<code>market_analysis</code>	<code>dict</code>	시장성·경쟁	없음	<code>summary</code> , <code>subdomain</code> , <code>market_size</code> , <code>growth</code> , <code>demand_drivers</code> , <code>competitors</code> , <code>differentiation</code> , <code>missing_info</code> 등
<code>scores</code>	<code>dict[str, float]</code>	투자 판단	없음	채점된 항목만의 1~5 점수 (<code>null</code> 항목은 빠짐)

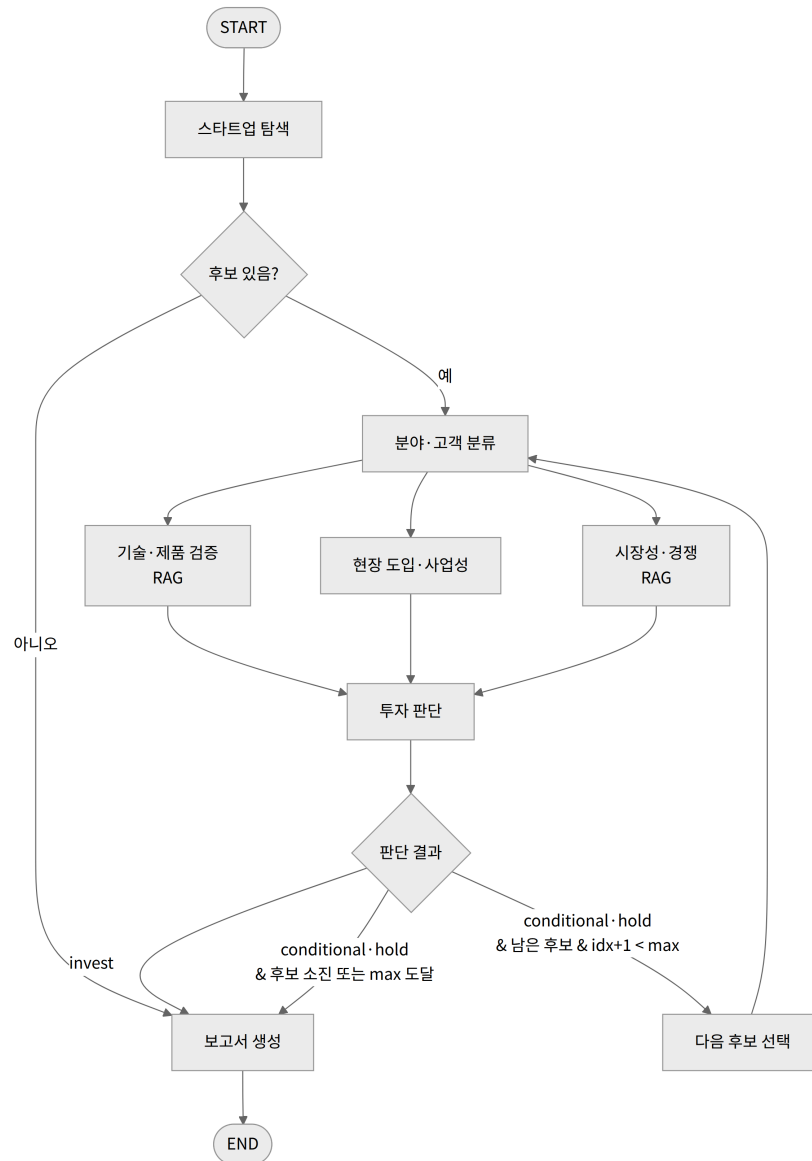
State 키	타입	생성·수정 Agent	Reducer	설명
investment_score	float None	투자 판단	없음	가중합 총점. 항목이 하나라도 비면 None
decision	Literal["invest", "conditional", "hold"]	투자 판단	없음	최종 판단. 필수 보류 조건 해당 시 점수와 무관하게 hold
decision_reason	str	투자 판단	없음	항목별 점수·근거·출처, 필수 보류 사유, 정보 부족 목록
evaluations	list[Evaluation]	투자 판단	operator.add	기업당 1건 누적: startup_name, score, decision, reason, score_details, blocking_risks, missing_information, source_ids
references	list[Evidence]	스타트업 탐색, 분야·고객 분류, 기술·제품 검증, 현장 도입·사업성, 시장성·경쟁	operator.add	분석에서 실제 인용한 자료만 누적: source_id, title, url, page, excerpt, published_at
report	str	보고서 생성	없음	생성된 투자 보고서 PDF의 절대 경로

- 쓰기 권한은 contracts.NODE_WRITES 에 Agent별로 정의되어 있고, 계약 밖 키를 쓰면 guard_node 가 실행 중 차단한다.
- 다음 후보 선택은 current_idx, selected_startup 만 바꾼다. 분석·점수 키는 다음 후보의 Agent가 다시 실행되며 덮어쓴다.
- references 는 후보를 넘어 누적되므로, 투자 판단은 현재 분석이 인용한 source_id 만 골라 쓰고 보고서는 인용한 출처만 REFERENCE에 실는다.

D-2. Graph 흐름 (개정)

메인 그래프에서 보완 재실행 분기를 빼고, 후보 없음 분기와 코드의 반복 조건을 넣었다.

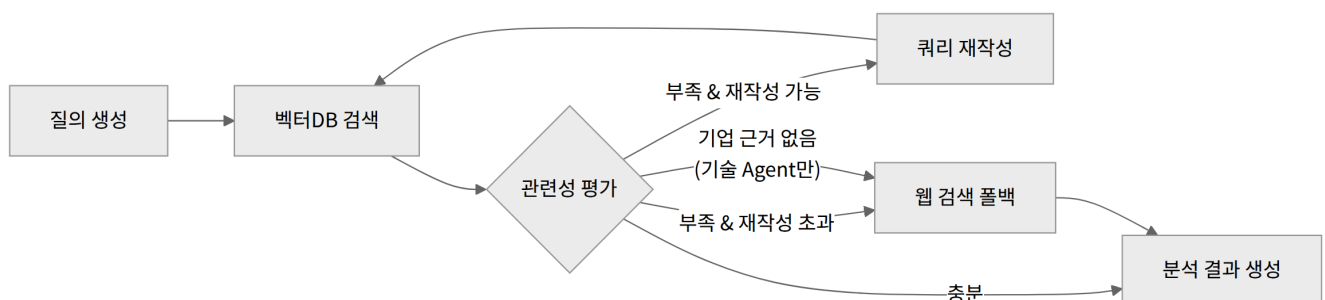
메인 그래프 (graph.py)



- 세 분석 Agent는 병렬로 실행되고, 투자 판단은 셋이 모두 끝난 뒤 한 번 실행된다(fan-in).

RAG 서브그래프 (CRAG)

시장성·경쟁은 공용 `crag.py` 의 LangGraph 서브그래프를 주제별로 병렬 실행하고, 기술·제품 검증은 같은 단계를 Agent 내부 루프로 구현했다. 기술 쪽은 기업 고유 자료가 없을 때도 웹으로 풀백한다(상세는 F 섹션).

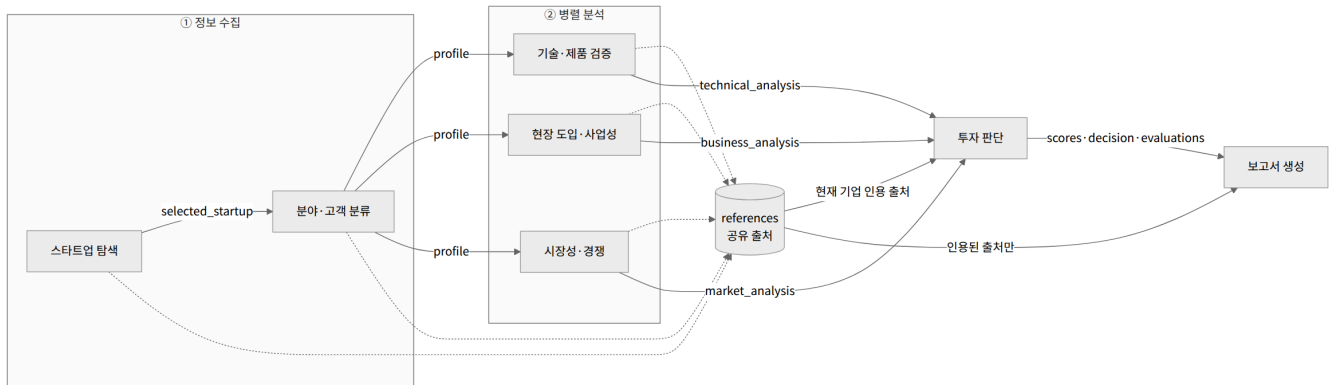


설정	기술·제품 검증 (technical.py)	시장성·경쟁 (crag.py)
구현 형태	Agent 내부 루프	LangGraph 서브그래프, 주제별 병렬
충분 기준	관련 자료 2개 이상 + LLM 충분 판정	관련 자료 2개 이상

설정	기술·제품 검증 (technical.py)	시장성·경쟁 (crag.py)
쿼리 재작성	최대 1회	최대 1회
웹 폴백 조건	재작성 후에도 부족, 또는 기업 고유 자료 없음	재작성 후에도 부족

에이전트 협업 구조

분석 Agent 셋은 각자의 분석 키와 공유 references에만 쓰고, 투자 판단은 세 분석 키와 references를 읽어 점수·판단을 쓴다. 이전 그림의 revision_request 흐름은 코드에 없어 뺐다.



D-3. Node·Edge 정의 (개정)

graph.py와 control.py 기준이다. 보완 재실행 조건과 초기화 항목은 코드에 없어 뺐다.

구분	이름 (코드 노드명)	실행 조건	처리 내용	다음 Node
Node	스타트업 탐색 (startup_discovery)	그래프 시작	선정 조건에 맞는 후보 목록 수집, 첫 번째 후보를 평가 대상으로 지정	후보 분기
Conditional Edge	후보 분기 (route_after_discovery)	탐색 완료	후보가 있으면 분류로, 없으면 보고서로	분야·고객 분류 또는 보고서 생성
Node	분야·고객 분류 (customer_profile)	후보 있음, 또는 다음 후보 선택	세부 분야, 비용을 지불하는 고객, 해결하려는 문제 정의	세 분석 Agent (병렬)
Edge	병렬 분기 (Fan-out)	분류 완료	profile을 세 분석 Agent에 동시에 전달	기술·제품 검증, 현장 도입·사업성, 시장성·경쟁
Node	기술·제품 검증 (technical_analysis)	분류 완료	Crag로 핵심 기술, 성능 근거, 업계 벤치마크, 기술적 한계 분석. 기업 고유 자료가 없으면 웹 폴백	투자 판단
Node	현장 도입·사업성 (business_analysis)	분류 완료	시연·시험 운영·유료 운영 구분, 고객 사례, 도입 비용·효과, 규제 리스크, 근거 수준 분석	투자 판단
Node	시장성·경쟁 (market_analysis)	분류 완료	주제별 Crag로 시장 규모·성장·수요 분석, 스타트업 기반 경쟁사 비교표 작성	투자 판단
Edge	결과 통합 (Fan-in)	세 분석 모두 완료	세 분석 결과와 출처를 투자 판단에 전달	투자 판단
Node	투자 판단 (investment_decision)	세 분석 모두 완료	인용 출처 대조, 6개 항목 채점(정보 부족은 null), Python 가중합, 필수 보류 확인, 판단 결정, 평가 이력 1건 추가	판단 결과 분기
Conditional Edge	판단 결과 분기 (route_after_decision)	투자 판단 완료	① invest → 보고서 생성 ② conditional·hold & 남은 후보 있음 & current_idx + 1 < max_iterations → 다음 후보 선택 ③ conditional·hold & (후보 소진 또는 반복 상한 도달) → 보고서 생성	다음 후보 선택 또는 보고서 생성
Node	다음 후보 선택 (next_candidate)	판단 결과 분기 ②	current_idx + 1, 다음 후보를 selected_startup으로 지정	분야·고객 분류

구분	이름 (코드 노드명)	실행 조건	처리 내용	다음 Node
Node	보고서 생성 (report_generation)	후보 없음, invest 결정, 또는 후보 소진·반복 상한 도달	평가 이력과 인용 출처로 5페이지 PDF 생성 (후보가 없으면 2페이지). PDF 경로를 report 에 저장	END

- **이전 설계서와의 차이:** 조건부 검토(conditional)는 바로 보고서로 가지 않고, 남은 후보가 있으면 다음 후보를 평가한다. 투자 추천 기업을 찾을 때까지 후보를 계속 보는 방식이다.

E. 투자 보고서 목차 (개정: 실제 PDF 구성)

보고서는 5페이지 PDF이며, report.py 가 페이지 수와 첫·마지막 제목(SUMMARY, REFERENCE)을 검증한 뒤 저장한다.

페이지	제목	내용
1	SUMMARY	대상 기업, 최종 판단·점수, 핵심 이유와 위험 요소. 반 페이지를 넘으면 오류
2	기업 및 사업 개요	기업 정보, 제품, 투자 단계, 창업자·핵심 인력, 비즈니스 모델
3	기술 및 시장 분석	핵심 기술과 구현 가능성, 시장 규모와 성장 전망, 경쟁사와 차별성
4	투자 평가 및 최종 의견	항목별 점수와 근거, 평가 이력(최대 10건), 투자 추천·조건부 검토·보류, 후속 확인 사항
5	REFERENCE	보고서 문단이 실제로 인용한 출처만 기재

- 점수·판단·기업 기본 정보는 LLM 문장이 아니라 State 값을 그대로 출력한다.
- 후보가 없으면 LLM 없이 SUMMARY와 REFERENCE로 된 2페이지를 만든다.
- 출력 위치는 output/pdf/ 이며 파일명에 UTC 시각을 넣어 구분한다.

F. 기술·제품 검증 Agent 상세 (추가)

담당 박주윤. 논문·보고서로 업계 기술 수준을 확인하고, 기업 고유 자료는 웹에서 보완해 기업 기술을 검증한다. 구현 agents/technical.py , 테스트 tests/test_technical.py , 적재 scripts/ingest_tech_docs.py , 상세 문서 docs/TECHNICAL_ANALYSIS.md .

처리 단계

1. **질의 생성:** 기업·분류 정보로 기술 개념 질의 3개를 만든다. 질의 하나에 개념 하나, 20단어 이내 자연어 문장이다.
2. **벡터DB 검색:** search_tech_docs 로 질의당 5개(MMR)를 찾아 중복을 제거한다.
3. **관련성 평가:** LLM이 자료별 관련 여부, 근거 충분성, 기업 고유 자료 여부(company_specific)를 판정한다.
4. **쿼리 재작성:** 문헌이 부족하면 동의어·영문 기술 용어로 1회 다시 검색한다.
5. **웹 폴백:** 재작성 후에도 부족하거나 기업 고유 자료가 없으면 기업명·제품명으로 Tavily를 2회 검색하고, 결과도 관련성 평가를 거친다.
6. **분석:** 근거 자료에 ID(S1, S2...)를 붙여 LLM이 ID로만 인용하게 하고, 결과를 실제 source_id 로 바꿔 저장한다.

출력 technical_analysis

키	내용
summary	기술 검증 요약
core_technologies	핵심 기술. layer 로 인식(perception)·판단(decision)·행동(action)·플랫폼(platform) 구분
differentiation	기술 차별성
performance_evidence	이 기업 제품의 성능 근거만
benchmark_context	논문·보고서가 보여 주는 업계 기술 수준 (다른 모델의 성능)
limitations	기술적 한계, 최소 1개
evidence_level	기업 기술의 검증 수준 (아래 표)
missing_items	확인하지 못한 항목
startup_name, retrieval	평가 대상, 사용 질의·재작성 횟수·웹 폴백 여부·관련·인용 자료 수

각 주장은 `statement`, `basis` (`company_claim` / `third_party` / `paper`), `source_ids` 를 가진다.

evidence_level	기준
high	제3자 자료(기사·고객 사례·인증)로 기업 제품 성능 확인
medium	기업 공개 자료로 기술 구성은 확인, 제3자 성능 근거 부족
low	기업 구체 근거 없이 업계 일반 문헌으로만 판단
insufficient	문헌·웹 모두 근거 없음. 투자 판단의 정보 부족 보류로 이어짐

환각 방지 장치

- 존재하지 않는 ID를 인용한 제3자·논문 주장은 버린다. 기업 주장(`company_claim`)만 출처 없이 남길 수 있다.
- 문헌·웹 모두 근거가 없으면 LLM 분석을 호출하지 않고 `insufficient` 를 반환한다.
- 논문·보고서(`tech`: 출처)만 인용했다면 LLM 판단과 무관하게 `evidence_level` 을 `low` 로 제한한다.
- `references` 에는 분석에서 실제 인용한 Evidence만 넣는다. 벡터DB가 비었거나 기업명이 없으면 빈 결과가 아니라 오류를 낸다.

설계 대비 확장 사항

항목	이전 설계서	구현	이유
웹 폴백 조건	문헌 관련성 부족 시	문헌 부족 또는 기업 고유 자료 없음	<code>tech_docs</code> 는 업계 일반 문헌이라 특정 기업을 검증할 근거가 없다. 이 조건이 없으면 논문만 보고 기업을 평가하게 된다
성능 근거	한 필드	<code>performance_evidence</code> / <code>benchmark_context</code> 분리	논문 속 다른 모델 성능이 기업 성능처럼 섞이는 문제 방지
Crag 형태	서브그래프	Agent 내부 루프	"재시도와 fallback은 Agent 내부에서 끝낸다"는 BASECODE 규칙. 단계 구성은 그림과 같다

검증 결과 (카본식스, 실제 API)

항목	초기 버전	최종 버전
검색 질의	"A AND B AND C..." 형태의 긴 검색식	개념 하나씩 담은 자연어 문장
웹 폴백	미실행	실행
인용 출처	논문·보고서 10건, 기업 자료 0건	기업 사이트·보도 5건 + 논문·보고서 7건
성능 근거	논문 속 다른 모델 성능이 섞임	기업 성능(시연 약 100회로 학습, 24시간 내 현장 투입)과 업계 벤치마크 분리
evidence_level	high (과대평가)	low (제3자 성능 수치 부재 반영)

- 테스트**: 단위 테스트 9개(가짜 LLM·검색으로 분기 검증) + 실제 API 스모크 테스트 4개(`RUN_LIVE_TESTS=1` 일 때만 실행).
- 실행 시간**: 기업 1곳당 LLM 5~6회 호출, 약 2~4분.

G. 미반영 항목과 후속 과제

가장 급한 것은 탐색·분류 Agent 복구다. 현재 master에서는 이 둘이 `NotImplementedError` 라 전체 그래프가 첫 노드에서 멈춘다.

우선순 위	항목	현재 상태	필요한 작업
1	스타트업 탐색·분야 분 류 구현	master의 <code>discovery.py</code> , <code>profile.py</code> 가 스 텝. 탐색 구현은 커밋 <code>b99ef26</code> 에 있음	구현 복구 후 master 병합, 전체 그래프 실행 확인
2	보완 재실행	이전 설계서에만 있음 (<code>revision_request</code> , <code>revision_count</code>)	구현하려면 <code>state.py</code> , <code>contracts.py</code> , <code>graph.py</code> , <code>decision.py</code> 와 세 분석 Agent를 함께 수정. 하지 않으면 현재 설계서대로 유지
3	tech_docs 임베딩 검증	미실시	<code>ingest.py</code> 의 <code>tech</code> 경로(<code>data/tech/raw</code> , <code>sources.json</code>)와 실제 위치(<code>data/tech_docs/</code> , <code>manifest.json</code>)를 맞춘 뒤 <code>retrieval_eval</code> score tech 실행

우선순 위	항목	현재 상태	필요한 작업
4	적재 방식 통일	기술 문서는 <code>scripts/ingest_tech_docs.py</code> , 시장 문서는 <code>ingest.py</code>	기술 문서도 <code>ingest.py</code> 로 옮겨 공용 정제(제어문자 제거, 섹션 경계 분할)를 적용
5	REFERENCE 중복	같은 논문·같은 쪽의 다른 청크가 따로 표시됨	보고서 생성에서 제목+쪽 기준으로 병합
6	기술 벤치마크 출처 혼 동	여러 논문 결과를 한 문장에 섞는 경우가 있음	기술 Agent 프롬프트에 "한 문장에 한 논문" 규칙 추가
7	실행 시간	기업 1곳당 분석 Agent별로 수 분	경량 모델 사용, 후보 수 <code>max_iterations</code> 조정

최종 점검

- ☒ Domain과 선정 이유를 작성했다.
- ☒ 에이전트 역할과 입출력을 정의했다. (코드 State 키 기준)
- ☒ RAG 적용 대상과 구성을 작성했다. (실제 적재 문서·Retriever 반영)
- ☒ 오픈소스 Embedding 모델과 선정 이유를 작성했다. (market_docs 검증 결과 포함)
- ☒ 투자 평가표의 가중치 합계가 100%이다.
- ☒ State 타입과 Reducer를 정의했다. (코드 기준 17개 키)
- ☒ Mermaid Graph와 조건 분기를 작성했다.
- ☒ 투자 보고서 목차 초안을 작성했다. (실제 PDF 구성)
- ☐ 제출 파일명을 확인했다.